

Assignment 2 : MA20268

Disclaimer: This solution to assignments is made (typed & thought upon) completely by hand and with full understanding of the content by me

– Shubham Krishan, 24MA10063

Constraints: Using only `numpy` for calculations and `matplotlib` for visualizations

Question 1:

1. Generate two vectors $u \in \mathbb{R}^{200}$ and $v \in \mathbb{R}^{200}$ sampled independently from a uniform random distribution on the range $[-5, 5]$.

Define the target variable $y \in \mathbb{R}^{200}$ by

$$y_i = \begin{cases} 1, & \text{if } u_i^2 + v_i^2 > 4, \\ 0, & \text{otherwise,} \end{cases} \quad i = 1, \dots, 200. \quad (1)$$

Here, y_i , u_i , and v_i denote the i -th entries of the vectors y , u , and v , respectively.

Now, define the following basis functions:

$$\begin{aligned} g_0(u_i, v_i) &= 1, & g_1(u_i, v_i) &= u_i, & g_2(u_i, v_i) &= v_i, \\ g_3(u_i, v_i) &= u_i^2, & g_4(u_i, v_i) &= v_i^2, & g_5(u_i, v_i) &= u_i v_i. \end{aligned} \quad (2)$$

Use the basis functions to define the matrix $C \in \mathbb{R}^{200 \times 6}$ where

$$C_i^j = g_j(u_i, v_i), \quad (3)$$

with i as the row index and j as the column index.

Use the least-squares method to predict the target variable y by learning the coefficient vector $w \in \mathbb{R}^6$ in the linear model

$$y \approx Cw. \quad (4)$$

Figure 1: Q1

Initializing $u, v \in \mathbb{R}^{200}$

```
import numpy as np

u = np.random.uniform(-5, 5, 200)
v = np.random.uniform(-5, 5, 200)
```

Now, the target variable $y \in \mathbb{R}^{200}$ as defined:

```
y = np.where(u**2 + v**2 > 4, 1, 0)
```

Given basis functions: $g_0(u_i, v_i) = 1$, $g_1(u_i, v_i) = u_i$, $g_2(u_i, v_i) = v_i$, $g_3(u_i, v_i) = u_i^2$, $g_4(u_i, v_i) = v_i^2$, $g_5(u_i, v_i) = v_i u_i$

```
def g1(ui, vi):
    return 1

def g2(ui, vi):
    return ui

def g3(ui, vi):
    return vi

def g4(ui, vi):
    return ui**2

def g5(ui, vi):
    return vi**2

def g6(ui, vi):
    return ui*vi

basis_func = {
    0: g1,
    1: g2,
    2: g3,
    3: g4,
    4: g5,
    5: g6
}
```

Defining matrix $C \in \mathbb{R}^{200 \times 6}$

```
c = np.zeros((200, 6))

for i in range(200):
    for j in range(6):
        c[i][j] = basis_func.get(j, lambda u,v: None)(u[i], v[i])
```

Training for $\tilde{y}$:

the model for this problem is:

$$\tilde{y}(u, v) = \sum_{i=0}^5 w_i g_i(u, v) = w_0 + w_1 u + w_2 v + w_3 u^2 + w_4 v^2 + w_5 uv$$

That is, $\mathbf{y} = C\mathbf{w}$

to use least squares method, define: $E(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^{200} (y_i - \tilde{y}_i)^2$

Now, our main goal is to minimize $E(\mathbf{w})$.

to get $\tilde{w}$ we solve the least squares problem $\min_w \|y - Cw\|^2$

```
w = np.linalg.lstsq(c, y, rcond=None)[0]
```

```
print("Weights: ")
for i, val in enumerate(w):
    print(i+1, val)
```

```
Weights:
1 0.5730681443403384
2 -0.001467047998826411
3 0.0011336238464902003
4 0.019372510001187084
5 0.016387654622781377
6 -0.0006485475967913499
```

Final views on the problem

The target set and basis functions used here trained the model $\tilde{y}$ to predict whether a point (u, v) lies outside the circle of radius '2' or not (defined by the target condition: $u^2 + v^2 > 4$). So, now we should be able to predict this using the approximated weights $\tilde{w}$.

```
def pred(u, v, w):
    features = np.array([1,u,v,u**2,v**2,u*v])
    y_hat = features @ w # dot product of given surface with w
    return y_hat

TEST_SIZE = 100
MAX_PRINT = 10
u_test = np.random.uniform(-5, 5, TEST_SIZE)
v_test = np.random.uniform(-5, 5, TEST_SIZE)
print("TESTING on random uniform test with size: ", TEST_SIZE )

correct = 0
threshold = np.mean(y)

for i in range(TEST_SIZE):
    y_hat = pred(u_test[i], v_test[i], w)
    prediction = 1 if y_hat > threshold else 0
    t = 1 if u_test[i]**2 + v_test[i]**2 > 4 else 0

    if prediction == t:
        correct += 1
    if i < MAX_PRINT:
        print(f"({u_test[i]:.2f}, {v_test[i]:.2f}) => model: {prediction} truth: {t}")
    elif i == MAX_PRINT:
        print("...")

print(f"points outside the circle in training data: {threshold}%") # shows skewness in the
→ random training dataset
print(f"accuracy: {(correct/TEST_SIZE)*100}%")
```

```
TESTING on random uniform test with size: 100
(-2.39, -4.17) => model: 1 truth: 1
(-1.83, 1.58) => model: 0 truth: 1
(0.79, 2.67) => model: 0 truth: 1
(-4.05, -3.50) => model: 1 truth: 1
(-4.09, 1.32) => model: 1 truth: 1
```

```
(-2.02, 1.87) => model: 0 truth: 1
(1.66, -3.58) => model: 0 truth: 1
(-3.15, -3.24) => model: 1 truth: 1
(1.57, -0.17) => model: 0 truth: 0
(4.65, 0.42) => model: 1 truth: 1
...
points outside the circle in training data: 0.865%
accuracy: 62.0%
```

Note: Since the dataset is not balanced (i.e., the proportion of class 1 labels is significantly higher than that of class 0), a threshold of 0.5 is not appropriate. As the model is obtained via least-squares regression and produces continuous outputs, a decision threshold is required for classification. In this case, we use the empirical mean of the training labels as the threshold to account for the class imbalance.

Question 2:

- Download the Fashion-MNIST dataset (both train and test sets, 28×28 images). Crop each image by 4 pixels from both rows and columns (resulting in $20 \times 20 = 400$ dimensions per vectorized image).

Let x_i be a sample in the train set and $c(x_i)$ be its true class label (0–9). Run 9 separate simulations comparing class 0 (T-shirt/top) against each of the other classes, where:

- $c(x_i) = 0 \Rightarrow \text{label } +1$

1

- $c(x_i) = k \ (k \neq 0) \Rightarrow \text{label } -1$

The 9 experiments are:

$$\begin{aligned} &0 \text{ vs } 1, \ 0 \text{ vs } 2, \ 0 \text{ vs } 3, \\ &0 \text{ vs } 4, \ 0 \text{ vs } 5, \ 0 \text{ vs } 6, \\ &0 \text{ vs } 7, \ 0 \text{ vs } 8, \ 0 \text{ vs } 9 \end{aligned} \tag{5}$$

For each experiment, use least squares to train a linear regression model $f(\cdot)$ on the binary train subset.

On the test set, predict:

$$\hat{c}(x_j) = \begin{cases} +1 & \text{if } f(x_j) > 0 \\ -1 & \text{otherwise} \end{cases} \tag{6}$$

Using ground truth test labels, compute the misclassification error for samples truly belonging to class 0 or the opposing class k .

New Things Learned:

Python code:

- `np.linalg.lstsq(A, b, rcond=None)` => solves the matrix linear equation directly: $A\mathbf{x} = b$
- `np.where(<condition>, <val_T>, <val_F>)` => allowed me to construct np vectors directly using conditions

Inference of problems:

Question 1:

- understood the differences in applications between regression and classification models
- got more familiar with terminologies like class labels
- understood the use of model and basis function in the QUERY => PREDICTION flow.